

- Let's try building up complexity
 - start with only one output (no fine/coarse)
 - "normal" recovery (no PID/inversion)

→ could be better tbh

1. put everything in a box w/global chassy ground (mitigate ground loops)
2. trigger, holdoff, ROI per peak
3. spectral purity curve + Pi trigger w/ ROI per peak
4. Interface w/Digilock Pi to hold recovery
 - probably fastest with digital GPIO → digital in

High-level logic (Arduino)

Main loop:

- initialization
 - general
 - wait for Pi to initialize peaks
 - need "handshake" in setup() + Pi side *
 - peak vals + locations
 - ↳ store in global variables
 - ↳ eventually add hold if Digilock not locked (via digital pin)
 - spectral purity curve? → function
 - ↳ send to Pi for display
- main loop:
 - find peaks for slower + xbeams
 - check + update status (via digital pins)
 - if feedback engaged (via digital pin)
 - rvm recovery if necessary
 - send trace + peak locations every so often

↓
Communication protocol

"[START] Command"

Commands: Peak Initialization → find good method*

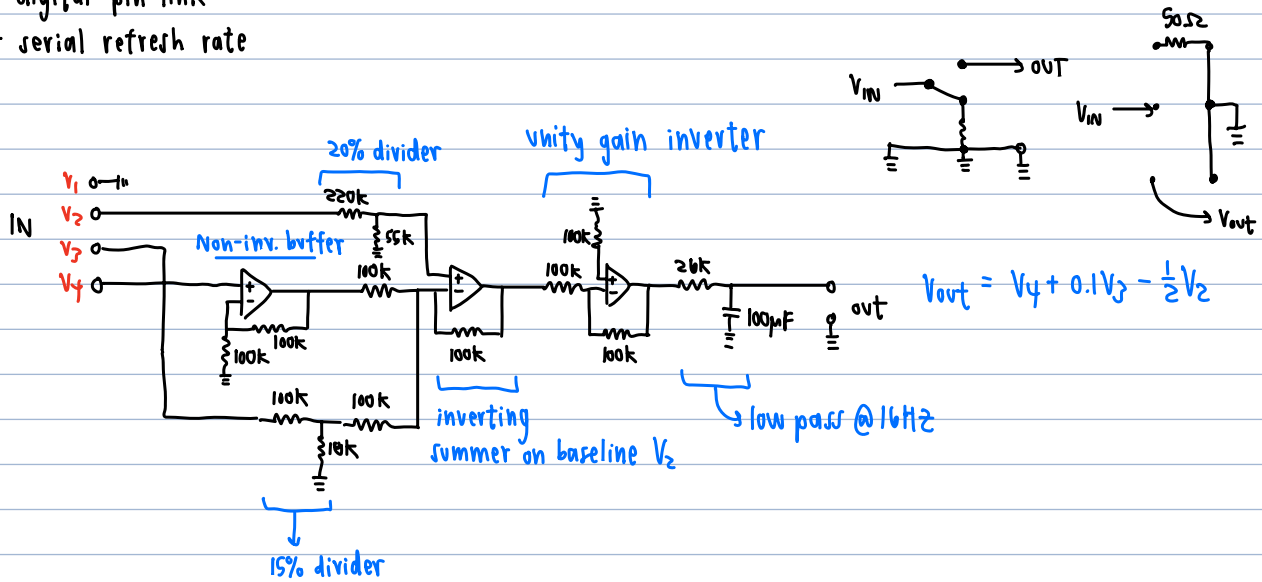
Trace → constant size

Peak Tracking → default

To-Do:

- test new communication w/print statements
- initialization handshake
- thread locking for sending commands
- FastAPI implementation
 - plus running-data class for timestamps + log(?)

- digital pin link
- serial refresh rate



$V_1 \rightarrow \text{gnd}$

$V_2 \rightarrow$ offset fine INVERTING i.e. +1V in gives $\sim -0.5V$ offset

$V_3 \rightarrow \text{sum} \rightarrow \text{tie to GND}$

$V_4 \rightarrow$ normal input

$\hookrightarrow$ this is likely a trim to get $\pm$ from V_{DAC}

$$I_{CO} = I_{leak} + I_{max} \cdot U_{mod}/10V, \quad I_{max} = \begin{cases} 200\mu A & \text{for slower} \\ 1A & \text{for xbeam} \end{cases}$$

conversion: $\frac{20\mu A}{V}$ slower

$\frac{100\mu A}{V}$ xbeam

$$V_{DAC} \sim 0.5V - 2.75V$$

Want $\sim \pm 2mA$ (4mA range) from 0.5 - 2.75 V_{in}

$$I_{mod} \approx (V_{DAC} - \frac{1}{2}V_{trim})\eta = (-1.15, 1.1)\eta$$

$$= \begin{cases} (-23, 22) mA & \text{slow} \rightarrow \text{too large!} \\ (-115, 110) mA & \text{xbeam} \rightarrow \text{too large!} \end{cases}$$

$\Rightarrow$ Tie V_4 to GND (?)

$$V_{out} = V_4 + 0.1V_3 - \frac{1}{2}V_2 \in \frac{(-2, 2) mA}{\eta}$$

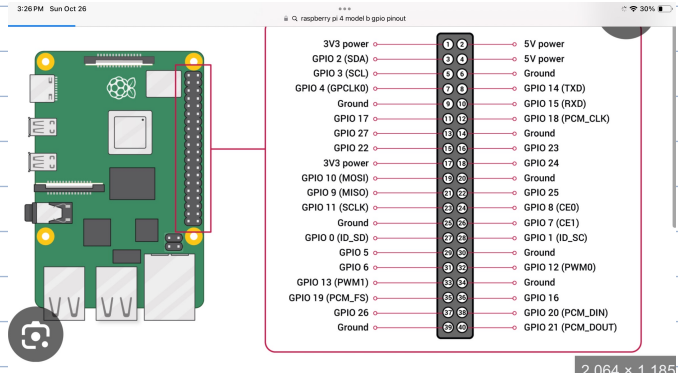
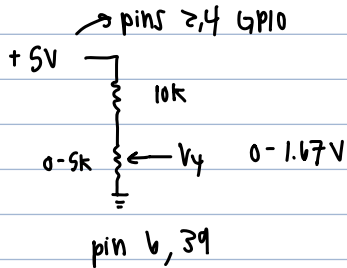
$$V_4 + 0.1V_3 \in \begin{cases} (1.05V, 1.25V) & \text{slower} \\ (1.13V, 1.17V) & \text{xbeam} \end{cases}$$

If V_4 coarse + ref through resistor, V_2 does fine

$$V_3 = V_{DAC} \in (0.5 - 2.75)$$

$$V_y^{slow} \approx 1V, V_y^{xbeam} \approx 1.08,$$

→ make this via trimpot set on Pi +5V



$$\Delta V_{DAC} = \frac{2.75 - 0.5V}{2^{12}} \approx 0.55 \frac{mV}{bit} = 0.055 mV_{out}/bit$$

$$V_{max} \approx 5mA = \eta \cdot V_{out}$$

$$slower: \frac{20mA}{V} \rightarrow V_{out,max} = 250mV = 4500 \rightarrow \text{no bound}$$

$$xbeam: \frac{100mA}{V} \rightarrow = 50mV = 1000 \rightarrow \text{bound}$$

$$step-slower = step-\mu A \cdot \frac{mV}{20\mu A} \cdot \frac{bit}{0.055 mV_{out}}$$

$$step-xbeam = \text{"} \cdot \frac{mV}{100\mu A} \cdot \frac{bit}{0.055 mV_{out}}$$

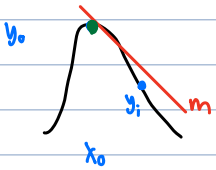
$$V_{DAC-slow} = I(mA) \cdot \underbrace{\frac{1V}{20mA} \cdot \frac{1000mV}{1V}}_{\substack{50 \text{ slow} \\ 10 \text{ xbeam}}} \cdot \frac{bit}{0.055 mV_{out}}$$

Spectral purity curve working!

To-Do:

- switches for output of current control ✓
- forward/backward SPC (hysteresis?) ✓
- check outputs for stability/noise ✓
 - xbeams seem "bouncy" ✓
- relock algorithm ✓

- slope calculation ✓
 - low vs gone: slower ✓ ^{test!} xbeam: ✓
 - ↳ try bumping slightly if low otherwise jump
 - running peak average? 0
 - ↳ does peak height change through the day?
 - auto-recalibration? 0
 - ↳ does SPC change through the day?
 - queue lock + wait for Arduino response ✓
 - Dashboard implementation of 1) SPC 2) relock ✓
 - Influx logging (Dash side)
 - digital pins
 - Digilock blue status
 - Lock status
 - Fail status
 - check dx sign w/ feedback params. ✓
 - error handling (both Serial+Pi/Dash) X
 - try getting rid of slope check on xbeam
- if running mean < Thresh: peak → slow loss
 OR running std > 3 σ → bouncing
- if running mean < 100 check val
- write lock last digital
 - jump by FWHM · feedback direction
 - calculate how far to come back from slope
 - recheck
- else
- bump vp by x amount
 ↳ determine from slope?
 - recheck it didn't make it worse
 if so, undo, try bumping down?
- else continue
- } LOST



$$y - y_0 = m(x - x_0)$$

$$y = m(x - x_0) + y_0$$

$$(x - x_0) = \frac{y - 0.95y_0}{m}$$

← safety factor?

Slower recovery logic:

Overall Structure

You have three tiers of recovery logic for the slower lock:

1.

Normal operation (FeedbackWrapper)

- Continuously checks running mean and std of `slowerRunningBuffer`.
- If below or unstable → triggers one of:
 - `bump_slower_up()` if "losing" (still near lock),
 - "lost" recovery path (larger bump + `relock_slower()`), or
 - `recover_slower()` if previous attempts flagged `slower_fail = true`.

2.

Intermediate recovery (`bump_slower_up`, `relock_slower`)

- `bump_slower_up()`: small incremental bumps up (main failure mode recovery).
 - Stops if:
 - Lock recovers (signal > threshold) → success,
 - Slope goes negative → peak overshoot → passes to `relock_slower`,
 - Signal drops too low → emergency jump back → passes to `relock_slower`.
- `relock_slower()`: iterative downward bumping to climb back onto the correct slope.
 - Recurse until success, max attempts, or total failure.

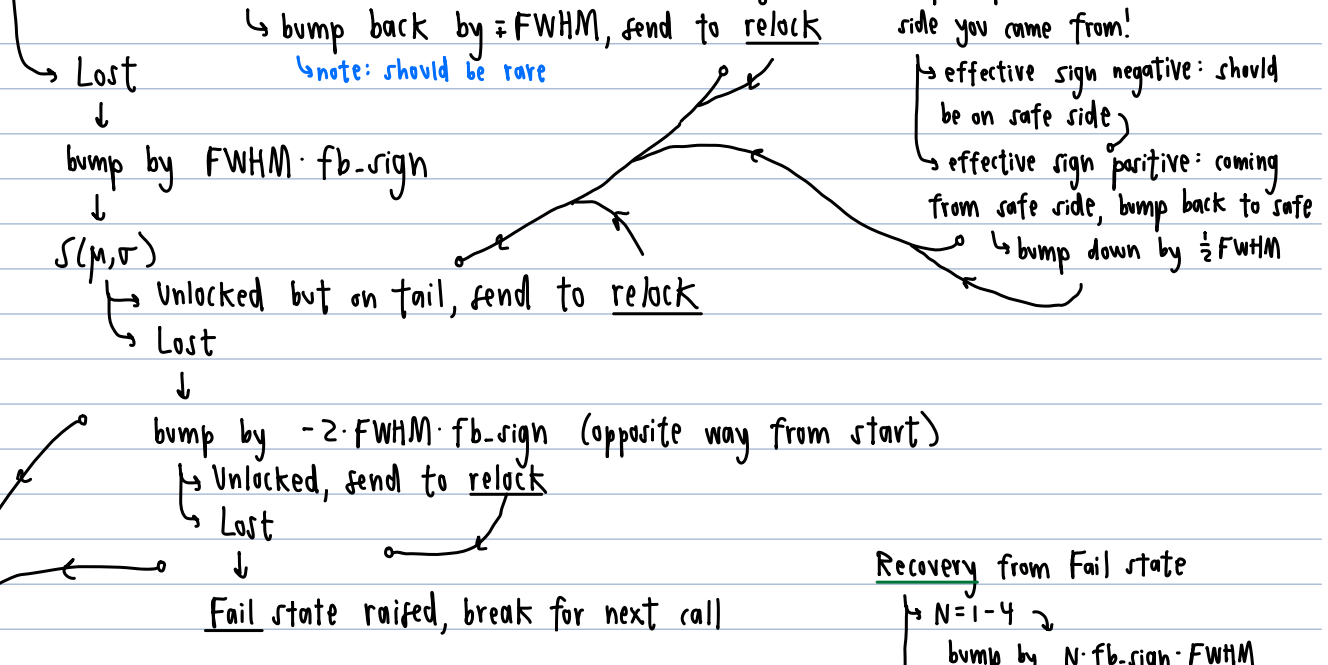
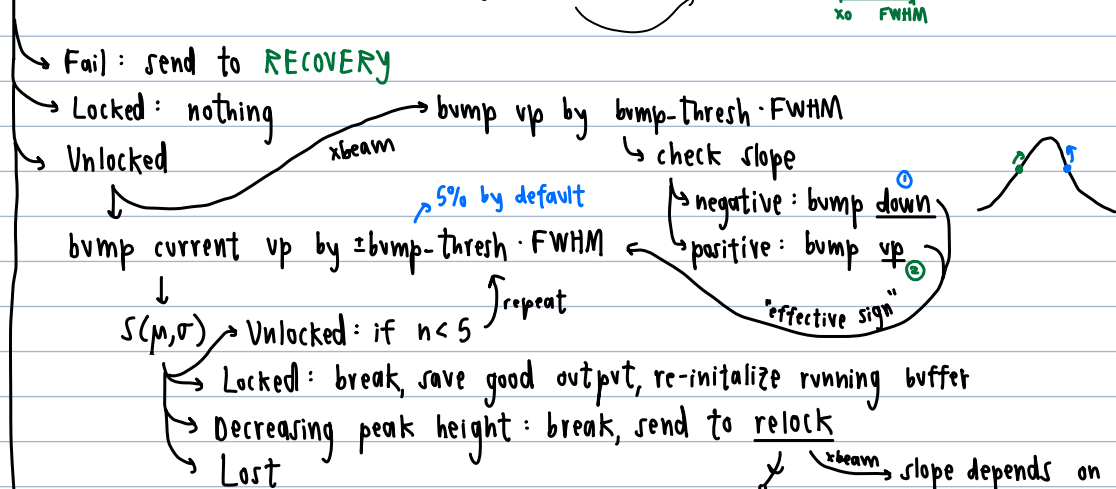
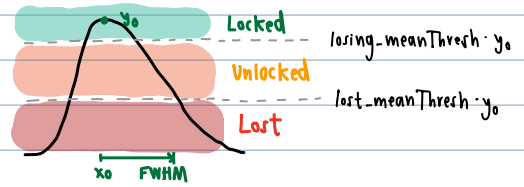
3. Full recovery (`recover_slower`)

- Triggered when all else fails (`slower_fail = true`).
- Jumps far above expected lock (4xFWHM) and scans down until a new peak is found.
- Terminates if peak found or bottom reached.

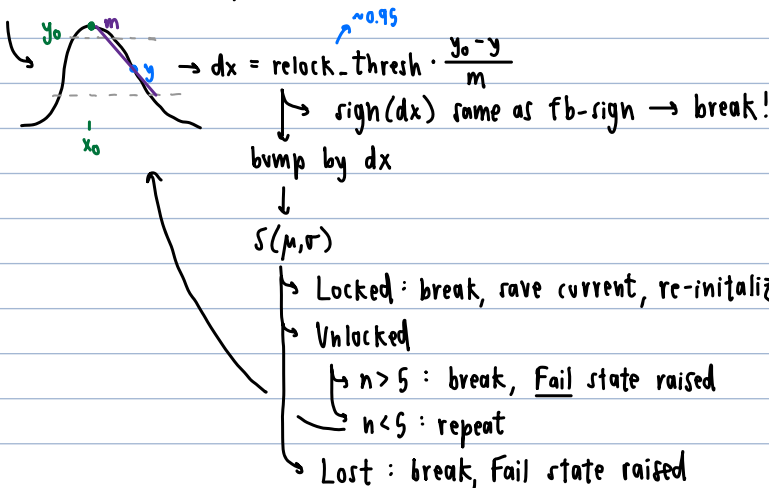
running mean μ , running std σ , fail state F



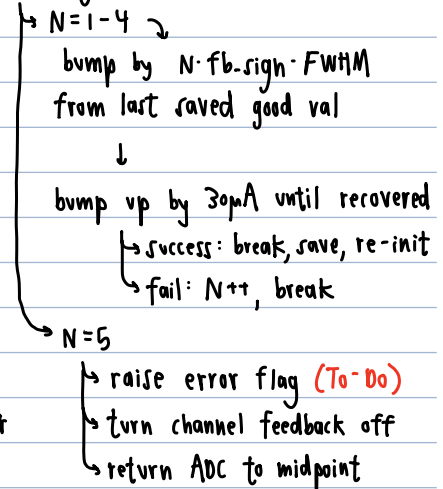
state check $S(\mu, \sigma) \in \{\text{Locked, Unlocking, Lost, Fail}\}$



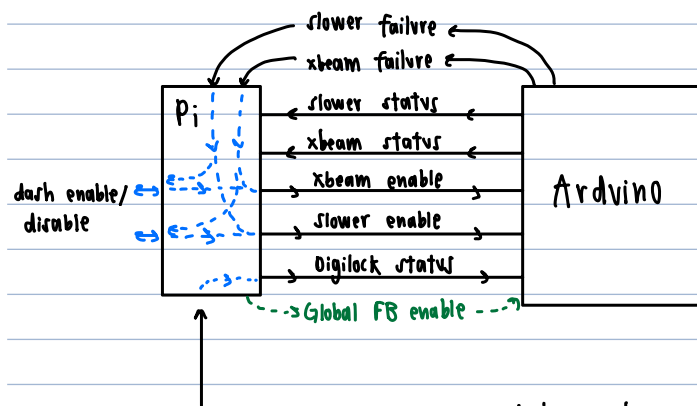
Relock → assumption: in unlocked state (but not lost)



Recovery from Fail state



Digital Pins



digilock status
check via Dash app globals

status gets sent to Dash for global LEDs

- green for locked
- orange for lost
- red for failure
- grey if global FB not enabled

if failure detected on Pi side

- enable turned off locally
- zero output via Z[S/X] over serial
- send failure to Dash
- turn off enable on that channel

	Pi	Arduino
Digilock status:	GPIO17 (11)	23
Slower enable:	GPIO27 (13)	24
Xbeam enable:	GPIO22 (15)	25
Slower status:	GPIO23 (16)	26
Xbeam status:	GPIO24 (18)	27
Slower failure:	GPIO5 (29)	28
Xbeam failure:	GPIO6 (31)	29

Update 11/03/25

- Most things working
- fix bug with states not reflecting lock status via GPIO → fixed?
- Bumping one manually kills the other? X
- Check fail logic on GPIO ↔ dash communication O
- Influx/Grafana logging ✓
- Master FP logging
- State refresh y_0 /peak height?

11/05/25

- Add "paused" state for injections while master isn't locked (Dash/Influx)
- Redo state detection Pi ↔ Dash
 - global state store?
- Add permissions
- Add slack alerts
- Check "locked" LED in digilock
- Check Doppler signal for drift
- Check if detected peak at window edge! O
 - ↳ auto-reset holdoff??
 - ↳ check noise on trigger pin?